SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code: CSA0309	Course Title: Data Structures
CO Assessed: CO2 – Manipulation (BL3, BL4)	Assessment: ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SO
Weightage: 30% of CIA	Total Marks: 100

CO2 Statement: Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.

Student Name: Pawan C	Reg. No.: 192521202
Section / Batch: IT – B	Date: 17/07/2026

Code of Conduct

I Pawan C / 192521202 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Pawan C*

Instructions

• This test contains 80 Analytical problem-solving questions. Total: 100 Marks.

• When a student answers using Analytical problem-solving, the evaluation should cover the following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.

• Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.

• No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50

GRAND TOTAL: 100 Marks

Annexure A – Analytical Problem Solving Answers

S.No: 28 Reg No: 192521202 Name: PAWAN C

Question 1

Insert the following elements 100, 200, 300, 400, 500 into the linked stack and pop the elements 100 and 200 from the linked stack. Show the linked structure before and after linked stack operations.

Answer:

A linked stack is a stack implemented using a singly linked list, where each node stores a data field and a pointer (next) to the node below it. The topmost node of the linked list always represents the TOP of the stack, and all insertions (Push) and deletions (Pop) happen only at this end, which gives the linked stack its Last-In-First-Out (LIFO) property.

Step 1 – Insertion (Push Operations): The elements are pushed one after another in the given order: 100, 200, 300, 400, 500. Each new element is inserted at the head of the linked list and becomes the new TOP, with its next pointer linked to the previous TOP node.

Structure after all five Push operations (TOP on the left):

TOP → 500 → 400 → 300 → 200 → 100 → NULL

Step 2 – Deletion (Pop Operations): Because a stack strictly follows LIFO, a Pop() operation can only remove the node currently at TOP; it is not possible to directly remove an element such as 100 or 200 that lies in the middle or bottom of the stack without first popping every element above it. Performing two Pop() operations on the stack above removes the two most recently pushed elements, i.e. 500 and then 400, in that order.

Structure after two Pop() operations:

TOP → 300 → 200 → 100 → NULL

To specifically remove 100 and 200, the stack must be popped down to those levels: three more Pop() operations remove 300, then 200, then 100 – which again confirms that 100 and 200 (pushed first) sit at the bottom of the stack and can only be reached after every element pushed after them has been removed. This demonstrates the core LIFO principle of a stack: the last element pushed is always the first one available to pop, and elements pushed earliest are the hardest to reach.

Time complexity: Push and Pop on a linked stack are both O(1), since only the head pointer is updated; no traversal is required.

Question 2

An online food delivery service receives the following orders: O1, O2, O3, O4. While O2 is being prepared, an express order O5 arrives. (a) Explain why a normal queue may not satisfy customer requirements. (b) Recommend an appropriate queue implementation and justify your choice.

Answer:

(a) Limitation of a normal (simple) queue: A simple queue is a First-In-First-Out (FIFO) structure, so orders are always served strictly in their arrival sequence: O1, O2, O3, O4, and finally O5. When express order O5 arrives while O2 is being prepared, a plain FIFO queue offers no mechanism to move O5 ahead of the already-waiting O3 and O4. This means an urgent/express order would be delayed behind two normal orders, which defeats the purpose of an ‘express’ service and reduces customer satisfaction and delivery efficiency.

(b) Recommended queue implementation – Priority Queue (or a two-level queue system): A Priority Queue serves elements based on an assigned priority value rather than pure arrival order. Each order is tagged with a priority (Express = high priority, Normal = low priority). New orders are inserted according to priority, and the dequeue operation always removes the highest-priority order currently waiting. In this scenario, once O2 finishes preparation, the priority queue would place O5 ahead of O3 and O4 because it has higher priority, even though it arrived later.

An equivalent and simpler alternative is a **multi-level (two-queue) system**: maintain a separate Express queue and a Normal queue, and always dequeue from the Express queue first if it is non-empty, otherwise serve from the Normal queue. This keeps insertion O(1) (unlike a heap-based priority queue, which is O(log n)) while still guaranteeing that express orders are served before normal orders, and normal orders among themselves still retain fairness (FIFO order within each sub-queue) so that no ordinary customer is starved indefinitely.

Justification: Either implementation preserves fairness for normal customers (FIFO within the same priority level) while guaranteeing express orders are never stuck behind lower-priority orders, directly resolving the shortcoming of a plain FIFO queue described in part (a).

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____